



# MediControl

Aplicación de Gestión de Medicamentos

Trabajo Práctico de Python — Introducción a la Algoritmia, UADE

Cristian Aostri Lopez · David Espinosa · Facundo Balmacena · Patricio Zermo

*Python | OpenAI GPT-5.5 | python-dotenv*

# Contenido de la presentación

Recorrido por el proyecto, de la idea al código y a la futura interfaz

1

## Descripción del proyecto

Qué es MediControl y cómo está estructurado

2

## Menú principal

9 opciones de navegación del programa

3

## Código: alta y validaciones

agregar\_medicamento() y pedir\_fecha()

4

## Código: toma y búsqueda

registrar\_toma() y buscar\_medicamento()

5

## Stock, vencimientos y días restantes

verificar\_alertas() y calcular\_dias\_restantes()

6

## Asistente IA con GPT-5.5

hablar\_con\_ia() y el loop principal

7

## El futuro: de la consola a la app

Mockups de la interfaz que viene

# ¿Qué es MediControl?

Programa de consola en Python que permite registrar, consultar y administrar los medicamentos del hogar. Funciona sin internet, sin registro y sin instalaciones adicionales.

*Pensado especialmente para adultos mayores y personas con enfermedades crónicas.*

9

opciones de menú

9

funciones principales

1

asistente IA integrado

## Librerías utilizadas

**`datetime`**

Manejo de fechas de inicio, término y vencimiento

**`openai`**

Conexión con GPT-5.5 para el asistente IA

**`dotenv`**

Carga segura de la API key desde .env

**`os`**

Acceso a variables de entorno del sistema

# Funciones del programa

Nueve funciones cubren todo el ciclo de vida de un medicamento



`agregar_medicamento()`

Alta de nuevo medicamento con validaciones



`mostrar_medicamentos()`

Lista todos los medicamentos registrados



`registrar_toma()`

Descuenta stock al registrar una toma



`pedir_fecha()`

Valida el formato y rango de las fechas ingresadas



`main()`

Arma el menú, captura la opción y deriva a cada función



`verificar_alertas()`

Detecta stock bajo y vencimientos próximos



`calcular_dias_restantes()`

Estima días de stock según dosis y frecuencia



`buscar_medicamento()`

Busca un medicamento por nombre



`hablar_con_ia()`

Chat con asistente médico GPT-5.5

# Menú principal — 9 opciones

El loop while se repite hasta elegir la opción 9 (Salir)

|   |                             |          |
|---|-----------------------------|----------|
| 1 | Agregar medicamento         | Alta     |
| 2 | Mostrar medicamentos        | Consulta |
| 3 | Registrar toma              | Acción   |
| 4 | Cargar umbral y vencimiento | Config   |
| 5 | Calcular días restantes     | Cálculo  |

|   |                                   |          |
|---|-----------------------------------|----------|
| 6 | Buscar medicamento por nombre     | Consulta |
| 7 | Verificar alertas de medicamentos | Consulta |
| 8 | Consultar asistente IA            | IA       |
| 9 | Salir del sistema                 | Salida   |

**Condición de salida:** `while opcion != "9"` — comparación de texto, sin uso de break

# agregar\_medicamento()

Registro de medicamentos con validaciones de fecha y stock

## Validaciones implementadas



Las fechas se validan con `pedir_fecha()`, que exige formato YYYY-MM-DD



El stock no puede ser 0 ni negativo — se repite el ingreso hasta que sea válido



Si el medicamento ya venció al cargarlo, el sistema avisa pero no bloquea



También se guarda el nombre del doctor que recetó el medicamento

```
def agregar_medicamento():
    nombre = input("Nombre del medicamento: ")
    dosis = int(input("Pastillas/ml por toma: "))
    frecuencia = int(input("Tomas diarias: "))
    fecha_inicio = pedir_fecha(
        "Fecha de inicio (YYYY-MM-DD): ")
    fecha_termino = pedir_fecha(...)

    while fecha_termino < fecha_inicio:
        print("Error: término anterior al inicio.")
        fecha_termino = pedir_fecha(...)

    stock = int(input("Stock total disponible: "))
    while stock <= 0:
        stock = int(input("Stock no puede ser 0..."))

    vencimiento = pedir_fecha(...)
    if vencimiento <= datetime.now():
        print("El medicamento ya ha vencido.")

    nombre_doctor = input("Doctor: ")
```

# Stock, vencimientos y días restantes

## verificar\_alertas()

```
def verificar_alertas(medicamentos,
                     umbral_stock, dias_vencimiento):
    hoy = datetime.now()
    for med in medicamentos:
        if med['stock'] < umbral_stock:
            print(f"Alerta: stock bajo de
                  {med['nombre']}")

        dias_faltantes = (
            med['vencimiento'] - hoy).days
        if (dias_faltantes < dias_vencimiento
            and dias_faltantes >= 0):
            print(f"{med['nombre']} vence
                  en {dias_faltantes} días")
```

## calcular\_dias\_restantes()

```
def calcular_dias_restantes(
    medicamentos):
    for med in medicamentos:
        if med['dosis'] > 0:
            total_dia = (
                med['dosis'] *
                med['frecuencia'])
            dias_restantes = (
                med['stock'] // total_dia)
            print(f"{med['nombre']} tiene
                  stock para ~{dias_restantes}
                  días")
        else:
            print("Error en la dosis")
```

Los parámetros umbral y dias\_venc se ingresan en la opción 4 del menú, dentro de main(), y se pasan directo a las funciones cuando se elige la opción 7.

# hablar\_con\_ia()

Asistente médico basado en GPT-5.5 con historial de conversación



Usa el modelo GPT-5.5 de OpenAI vía la API Responses



La API key se carga desde un archivo .env, nunca en el código



El rol de sistema limita al asistente a medicamentos (Vademécum Argentino) con un guardrail explícito



El historial completo se envía en cada llamada para mantener contexto

*El loop usa "while mensaje.lower() != 'salir'" — sin break, condición directa en el while.*

```
def hablar_con_ia():
    historial = [{
        "role": "system",
        "content": ("Asistente médico,"
                    " solo VADEMECUM"
                    " ARGENTINO. No otros temas.")
    }]

    mensaje = ""
    while mensaje.lower() != "salir":
        mensaje = input("Tu: ").strip()
        if mensaje.lower() != "salir":
            historial.append(
                {"role": "user", "content": mensaje})

        response = client.responses.create(
            model="gpt-5.5",
            input=historial)

        respuesta = response.output_text

    print(f"IA: {respuesta}")
```



# main() — Loop principal

El corazón del programa: arma el menú y deriva a cada función

```
def main():
    medicamentos = []
    opcion = ""

    while opcion != "9":
        print("--- MENU PRINCIPAL ---")
        print("1. Agregar medicamento")
        ...
        print("9. Salir")
        opcion = input("Seleccione opción: ")

    if opcion == "1":
        medicamentos.append(
            agregar_medimento())
    elif opcion == "4":
        umbral = int(input("Umbral: "))
    elif opcion == "7":
        verificar_alertas(medicamentos, umbral, dias_venc)
    elif opcion == "9":
        print("Saliendo del sistema...")
    else:
```

## Conclusión

El proyecto demuestra cómo los conceptos básicos de programación pueden resolver un problema concreto y de alto impacto cotidiano: llevar un registro organizado de la medicación.

A través de listas, funciones y estructuras de control, el programa gestiona información de manera eficiente, valida los datos ingresados y genera alertas preventivas que pueden marcar una diferencia real en la salud de los usuarios.

**Hoy: todo corre 100% en consola, sin interfaz gráfica.**



# De la consola al celular

Hoy MediControl funciona 100% por código y línea de comandos.  
A continuación, una mirada a cómo podría verse mañana como aplicación.

**Próximo paso: una interfaz visual**

# Mockup: pantalla principal

Propuesta de interfaz — el código hoy, la app mañana



## Panel de inicio

Acceso directo a las 9 opciones del menú, traducidas a botones.



## Indicadores en vivo

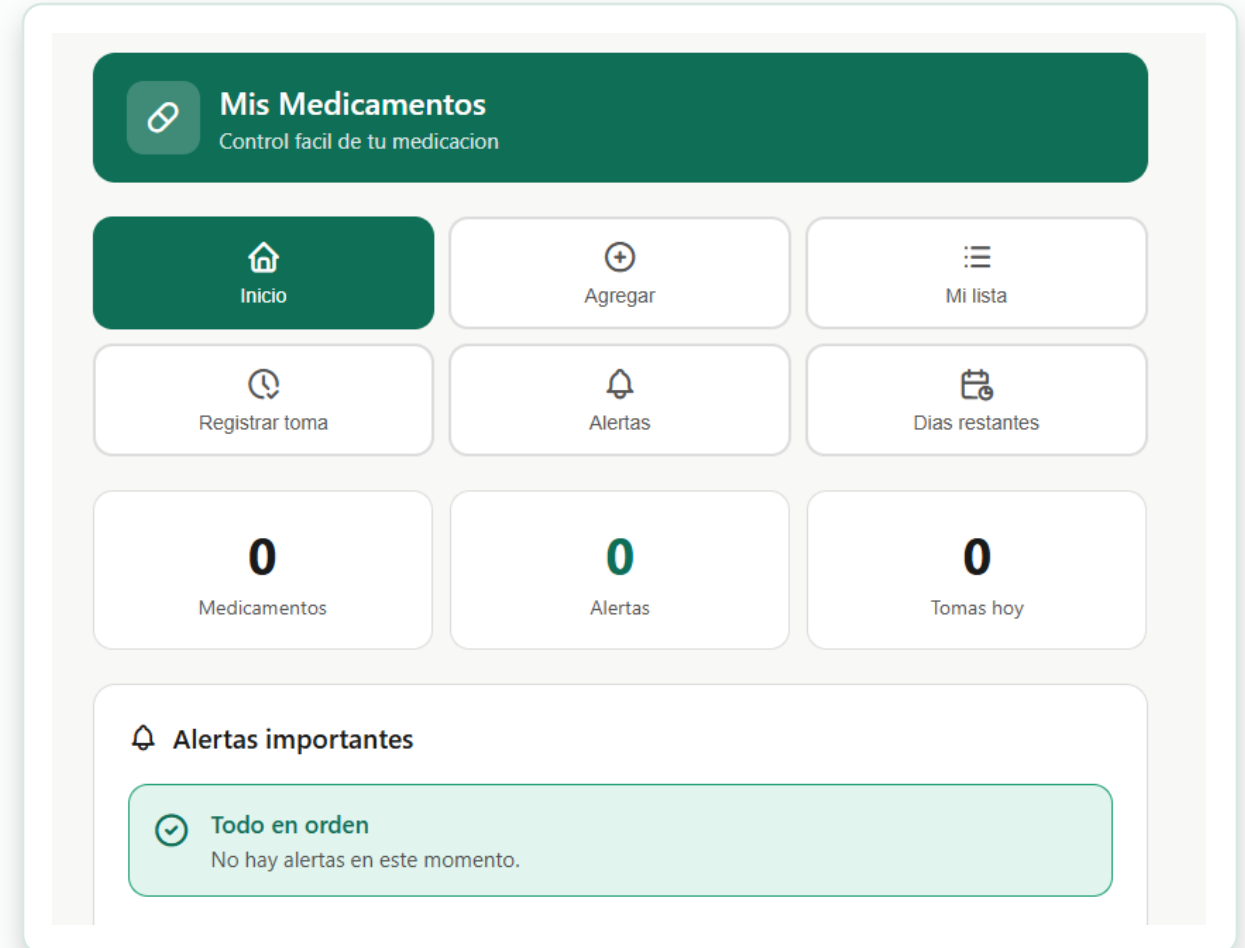
Medicamentos, alertas y tomas del día, igual que hoy devuelve la consola.



## Alertas importantes

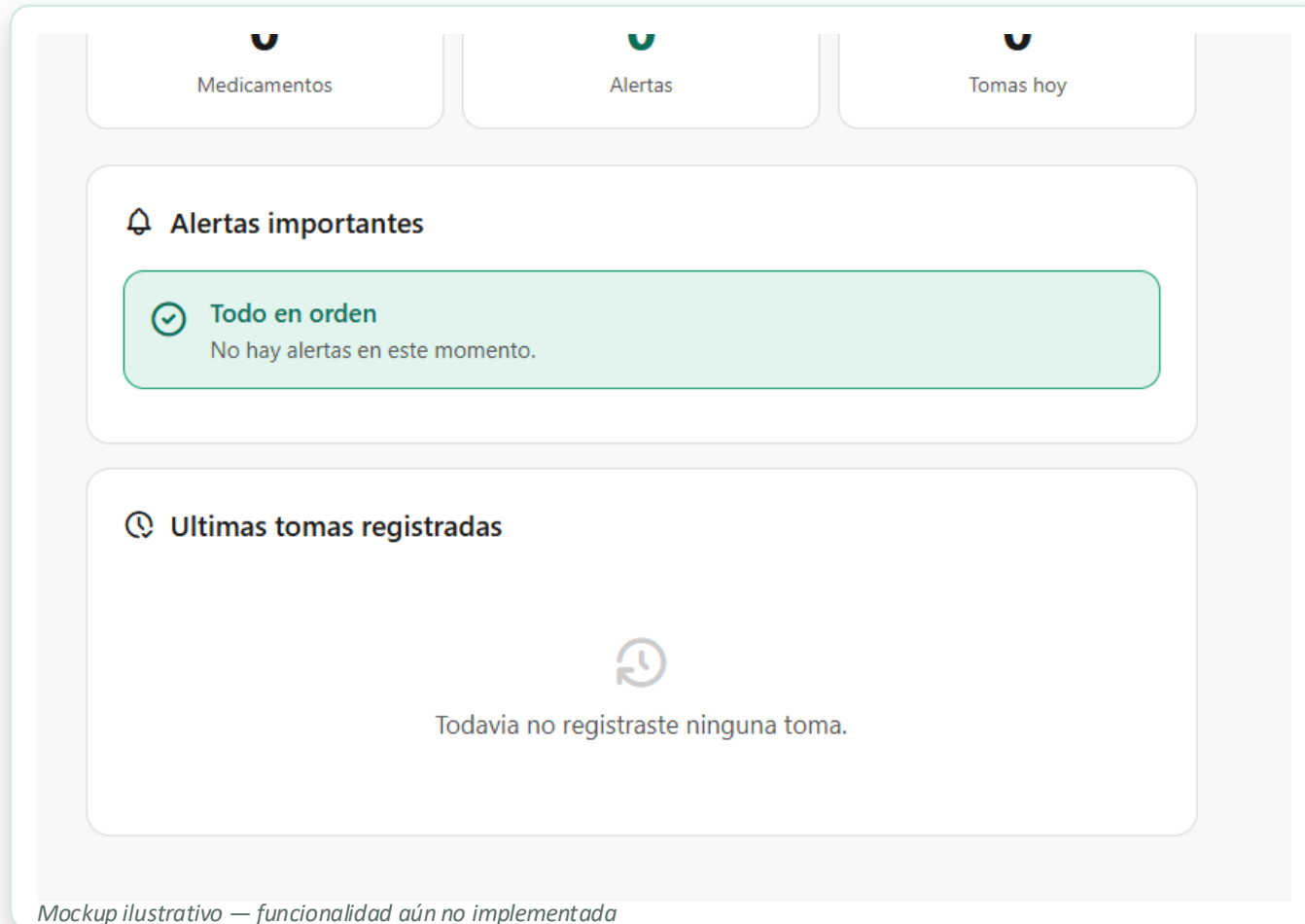
El mismo concepto de verificar\_alertas(), ahora visible de un vistazo.

Mockup ilustrativo — funcionalidad aún no implementada



# Mockup: alertas e historial

El mismo motor de datos, presentado de forma visual



## Alertas importantes

Traduce `verificar_alertas()` a un estado visual: "Todo en orden" o avisos puntuales.



## Últimas tomas registradas

Historial de `registrar_toma()`, hoy solo impreso por consola línea a línea.



## Próximo paso natural

Conectar esta vista con el asistente `hablar_con_ia()` para consultas rápidas.

# Próximos pasos posibles

De script de consola a producto completo



## Hoy

Programa de consola en Python, 100% funcional, sin interfaz gráfica



## Interfaz visual

Convertir el front mostrado en una app real (web o mobile)



## Persistencia

Guardar los datos en una base en vez de listas en memoria



## Multiusuario

Perfiles por paciente, útil para cuidadores y familias



# Gracias

MediControl — Gestión de Medicamentos

Cristian Aostri Lopez · David Espinosa · Facundo Balmacena · Patricio Zermo

UADE — Introducción a la Algoritmia — 2025